

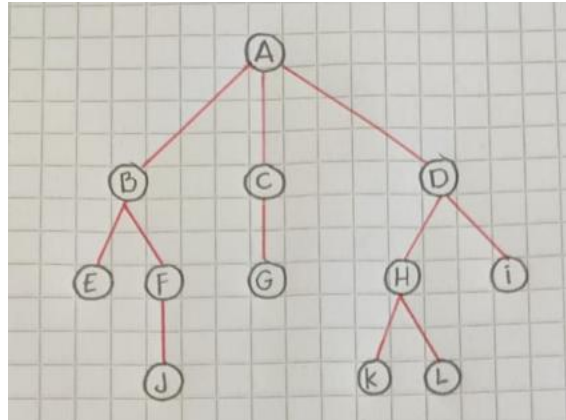
Taller de árboles, recorridos y complejidad computacional

Estructuras de datos para el análisis sintáctico descendente

Julián Beltran
Santiago Ortegón
Juan Orjuela

Punto 1. Conceptos y representación de árboles

1.1. Dibujo del árbol



1.2. Raíz, hojas, nodos internos y relaciones

Raíz: A. Es el único nodo que no aparece como hijo de nadie en la tabla, es decir, no tiene padre.

Hojas: E, G, I, J, K y L. Son seis y ninguna aparece en la columna de nodos padre, o sea que su grado es 0.

Nodos internos: A, B, C, D, F y H. Son los que sí tienen al menos un hijo. Vale la pena aclarar que la raíz también cuenta como nodo interno aquí porque tiene hijos.

Sobre las relaciones que pide el punto:

- **Padre de J:** F. Es el único nodo que tiene a J colgando de él.
- **Ancestros de L:** H, D y A. Los ancestros se obtienen subiendo desde L hasta la raíz: L viene de H, H viene de D y D viene de A. Por eso son tres.
- **Descendientes de B:** E, F y J. No son solo los hijos directos de B, sino todo lo que cuelga debajo de él: E y F son hijos, y J es hijo de F, así que también es descendiente de B.
- **Hermanos de H:** I. H e I comparten el mismo padre, que es D. K y L no son hermanos de H sino sus hijos.

1.3. Grados, profundidades y altura

El grado de un nodo es simplemente cuántos hijos tiene:

Nodo	A	B	C	D	E	F	G	H	I	J	K	L
Grado	3	2	1	2	0	1	0	2	0	0	0	0

Grado del árbol: 3, porque el grado del árbol es el mayor grado de todos sus nodos y ese es el de A, que tiene tres hijos.

Para las profundidades:

Nodo	Camino desde la raíz	Profundidad
A	A	0
F	A → B → F	2

J	$A \rightarrow B \rightarrow F \rightarrow J$	3
L	$A \rightarrow D \rightarrow H \rightarrow L$	3

Altura del árbol: 3. La altura es la profundidad máxima que alcanza alguna hoja, y las hojas más profundas son J, K y L, que están en el nivel 3. Si se midiera en número de niveles en vez de aristas serían 4, pero yo trabajo con la medida en aristas y la mantengo igual en todo el taller para que el código y el documento den lo mismo.

1.4. ¿Es binario, completo y balanceado?

¿Es binario? No. Un árbol binario exige que ningún nodo tenga más de dos hijos, y A tiene tres (B, C y D). Con ese solo nodo se rompe la definición, así que el árbol es general, no binario.

¿Es completo? No. Un árbol completo necesita que todos sus niveles estén llenos. Como el grado del árbol es 3, para estar lleno el nivel 2 tendría que tener 9 nodos y solo tiene 5 (E, F, G, H, I), y el nivel 3 tendría que tener 27 y solo tiene 3 (J, K, L). Además hay ramas que se cortan antes: E, G e I no tienen hijos mientras que sus vecinos sí. Entonces queda muy lejos de ser completo.

¿Es balanceado? Sí. El criterio que usamos es el de balanceo por altura: un árbol está balanceado si para cada nodo las alturas de los subárboles de sus hijos no se diferencian en más de 1, así

- **En A:** los subárboles de B, C y D tienen altura 2, 1 y 2. La diferencia máxima es 1.
- **En B:** E tiene altura 0 y F tiene altura 1. Diferencia de 1.
- **En D:** H tiene altura 1 e I tiene altura 0. Diferencia de 1.
- **En H:** K y L tienen altura 0 los dos. Diferencia de 0.
- **En C y F:** solo tienen un hijo, de altura 0. Si se cuenta el subárbol que falta como altura -1, la diferencia sigue siendo 1.

También se puede verificar por el lado de las hojas: todas están en el nivel 2 o en el nivel 3, que son niveles contiguos, así que ninguna rama es mucho más larga que otra. Con cualquiera de las dos formas de mirarlo el árbol queda balanceado.

1.5. Análisis de complejidad

Las tres operaciones que menciona el punto (contar hojas, calcular la altura y buscar un valor que no está) tienen que ninguna puede saltarse ningún nodo.

Contar las hojas es $\Theta(n)$. Para saber si un nodo es hoja hay que mirarlo, y no hay ningún atajo que permita descartar un subárbol completo, porque cualquier subárbol puede contener hojas. Toca visitar los 12 nodos.

Calcular la altura es $\Theta(n)$. La altura la define la rama más larga, y no se sabe cuál es sin bajar por todas. En cada nodo el trabajo extra es un máximo entre las alturas de los hijos, que es $O(\text{grado})$; sumando sobre todos los nodos eso equivale a recorrer cada arista una vez, o sea $O(n)$.

Buscar un valor que no existe es $\Theta(n)$. Este es justo el peor caso de una búsqueda: como el árbol no está ordenado, no se puede descartar ninguna rama por anticipado y hay que agotar el árbol entero antes de poder afirmar que el valor no está. Es distinto de buscar un valor que sí existe, donde el recorrido puede cortarse apenas lo encuentra: en el mejor caso, si está en la raíz, es $O(1)$.

En memoria, un recorrido recursivo gasta $O(h)$, porque la pila de llamadas solo guarda el camino que va desde la raíz hasta el nodo actual. En este árbol $h = 3$, así que es un gasto mínimo.

Punto 2. Construcción de un árbol de expresiones

La expresión que hay que representar es

$$(a + 3) \times (b - 2) + \frac{c}{4}$$

2.1. Operandos y operadores

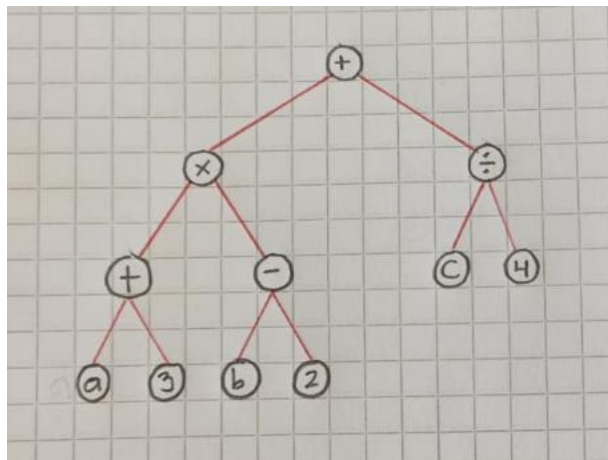
Operandos (6): a, 3, b, 2, c y 4. Son los valores sobre los que se opera y en el árbol siempre quedan en las hojas, porque no tienen nada colgando debajo.

Operadores (5): el + de dentro del primer paréntesis, el - del segundo paréntesis, el × que los multiplica, el ÷ de la fracción c/4 y el + que une todo. Los operadores son siempre nodos internos porque necesitan hijos, que son sus operandos.

Las reglas que hay que respetar para armar el árbol son las de siempre: los paréntesis mandan primero, después × y ÷ (que tienen la misma precedencia), y de últimos + y -. Entre operadores de la misma precedencia se asocia de izquierda a derecha.

2.2. Árbol de la expresión

La forma de armarlo es al revés de cómo se evalúa, el operador que se ejecuta de último es el que queda en la raíz, porque es el que parte la expresión en dos mitades. Aquí ese es el + de afuera, ya que es el de menor precedencia y no está dentro de ningún paréntesis. A la izquierda queda toda la multiplicación y a la derecha la división. Después repito el mismo razonamiento dentro de cada mitad hasta llegar a los operandos sueltos.



Expresión de $(a + 3) \times (b - 2) + c/4$.

2.3. Recorridos

Recorrido	Resultado
Preorden	+ × + a 3 - b 2 ÷ c 4
Inorden	a + 3 × b - 2 + c ÷ 4
Postorden	a 3 + b 2 - × c 4 ÷ +

Algo a tener en cuenta en el inorden es que así como queda, plano, la expresión cambió de significado, porque se perdieron los paréntesis y ahora parecería que el $\times$ solo afecta al 3 y a la b. Para que el inorden sea fiel hay que ir poniendo paréntesis cada vez que un hijo tiene un operador de menor precedencia que su padre, y así se recupera la expresión original:

$$((a + 3) \times (b - 2)) + (c / 4)$$

2.4. Relación con las notaciones

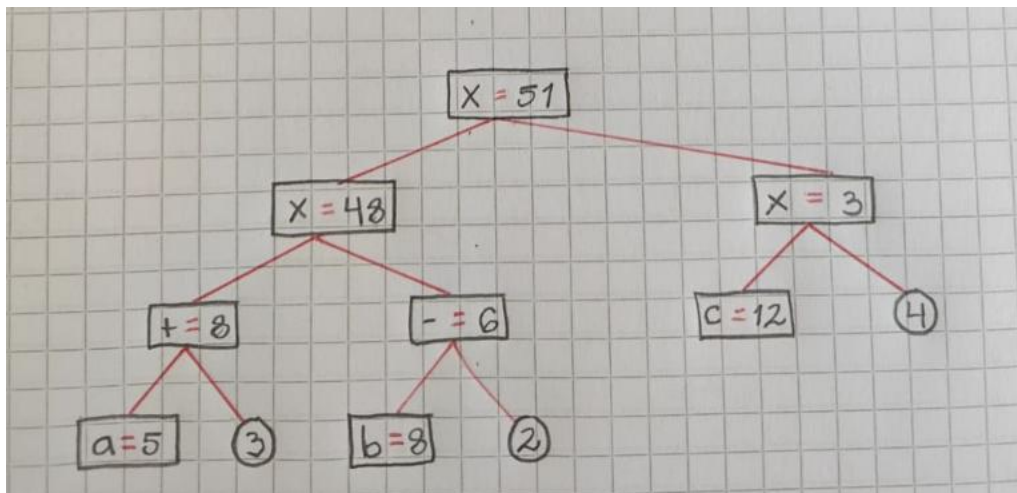
Recorrido	Notación que produce	Característica
Preorden	Prefija	El operador va antes de sus operandos. No necesita paréntesis.
Inorden	Infija	Es la notación de siempre, pero es la única que necesita paréntesis.
Postorden	Postfija	El operador va después. No necesita paréntesis y es la que usan las pilas.

La prefija y la postfija no son ambiguas sin paréntesis, mientras que la infija sí lo es. Eso pasa porque en esas dos el orden de los símbolos ya contiene la estructura del árbol, y en la infija no.

2.5. Evaluación con $a = 5$, $b = 8$ y $c = 12$

La evaluación la hice recorriendo el árbol en postorden, no reemplazando en la expresión original. La idea es que cada nodo espera a que sus dos hijos ya sean un número y ahí sí aplica su operador:

Paso	Nodo visitado	Qué hace	Valor que devuelve
1	a	hoja: devuelve el valor de la variable	5
2	3	hoja: devuelve la constante	3
3	+	aplica $5 + 3$ con lo que devolvieron sus hijos	8
4	b	hoja	8
5	2	hoja	2
6	-	aplica $8 - 2$	6
7	$\times$	aplica 8×6 (los dos subárboles ya están resueltos)	48
8	c	hoja	12
9	4	hoja	4
10	$\div$	aplica $12 \div 4$	3
11	+ (raíz)	aplica $48 + 3$ y termina	51



El mismo árbol con el valor que devuelve cada nodo. El resultado final es 51.

El resultado es 51. Comprobando por fuera del árbol: $(5 + 3) \times (8 - 2) + 12/4 = 8 \times 6 + 3 = 48 + 3 = 51$, así que el recorrido dio lo mismo.

2.6. Preguntas de análisis

1. ¿Por qué la evaluación puede hacerse en postorden? Porque un operador no puede aplicarse hasta que sus dos operandos sean valores concretos, y el postorden garantiza exactamente eso, visita un nodo solo después de haber visitado todos sus descendientes. Cuando llego al nodo x , ya pasé por el $+$ y por el $-$, o sea que sus dos hijos ya son 8 y 6. Con preorden no funcionaría, porque llegaría al x cuando todavía no sé cuánto valen sus ramas. Es el mismo principio de las calculadoras de notación polaca inversa: se van apilando operandos y cuando aparece un operador se sacan los dos últimos.

2. ¿Cuál es la complejidad temporal de evaluar el árbol? $O(n)$, donde n es el número de nodos. Cada nodo se visita una sola vez y el trabajo que se hace en él es una operación aritmética, que es $O(1)$. En este árbol son 11 visitas.

3. ¿Cuál es la complejidad espacial del recorrido recursivo en función de h ? $O(h)$. La memoria que consume no depende de cuántos nodos tenga el árbol sino de qué tan profundo esté el nodo que estoy procesando, porque la pila de llamadas solo guarda las llamadas del camino actual desde la raíz. Aquí $h = 3$, así que en el peor momento hay cuatro llamadas apiladas.

4. ¿Qué pasa con la memoria si el árbol está completamente desbalanceado? Que h deja de ser pequeño y se vuelve igual a $n - 1$: el árbol degenera en una lista y cada nodo tiene un solo hijo. Ahí $O(h)$ se convierte en $O(n)$ y la recursión pasa a guardar un marco de pila por cada nodo del árbol. Con entradas grandes eso termina en un desbordamiento de pila (stack overflow), que en Python aparece como RecursionError alrededor de los mil niveles. Es la razón práctica por la que a veces conviene la versión iterativa con pila propia: esa pila vive en el heap y no tiene ese límite tan bajo. Esto no es un caso teórico en un compilador: una expresión como $a+a+a+a+\dots$ encadenada produce justamente un árbol así de desbalanceado.

Punto 3. Recorridos en profundidad: DFS

3.1. La estructura de datos

Como el árbol del punto 1 tiene un nodo con tres hijos, no me servía una estructura binaria con punteros izquierdo y derecho. Usamos un árbol general, cada nodo guarda un valor y una lista con las referencias a sus hijos. Con una lista da lo mismo que un nodo tenga cero, dos o diez hijos, y los recorridos quedan escritos como un ciclo sobre esa lista en vez de dos llamadas fijas.

```
class Nodo:
    def __init__(self, valor):
        self.valor = valor
        self.hijos = []          # lista de objetos Nodo

    def agregar(self, hijo):
        self.hijos.append(hijo)
        return hijo

    def es_hoja(self):
        return len(self.hijos) == 0
```

Para armar el árbol del punto 1 creamos primero los doce nodos en un diccionario y después los enlazo usando la misma tabla padre-hijo del enunciado, así no me pierdo con las referencias.

3.2. Los cinco algoritmos

DFS recursivo en preorden. Es el más corto porque la recursión hace sola el trabajo de la pila: cada llamada que queda pendiente es un nodo al que todavía tengo que volver. Primero agregamos el valor del nodo actual y después bajo por cada hijo en orden.

```
def dfs_preorden_recursivo(nodo, visitados=None):
    if visitados is None:
        visitados = []
    if nodo is None:
        return visitados
    visitados.append(nodo.valor)          # visito el padre (PRE-orden)
    for hijo in nodo.hijos:              # y luego bajo por cada hijo
        dfs_preorden_recursivo(hijo, visitados)
    return visitados
```

DFS iterativo con pila. Hace exactamente lo mismo pero manejando yo la pila. El detalle que me costó notar es que hay que apilar los hijos al revés: como la pila es LIFO, si los apilo en el orden normal el primero que sale es el último hijo y el recorrido queda invertido respecto al recursivo. Con reversed() los dos dan idéntico, y eso lo verifico en la salida del programa.

```
def dfs_preorden_iterativo(raiz):
    if raiz is None:
        return []
    visitados = []
    pila = [raiz]
    while pila:
        actual = pila.pop()              # saco el ultimo que entro
        visitados.append(actual.valor)
        for hijo in reversed(actual.hijos): # para que el primer hijo quede arriba
            pila.append(hijo)
    return visitados
```

Búsqueda de un valor. Es un preorden que se corta apenas encuentra el valor. Le agregamos un contador para poder reportar cuántos nodos tuvo que mirar, que es lo que hace visible la diferencia entre el mejor y el peor caso. Usamos una lista de un solo elemento como contador porque así el valor sobrevive a las llamadas recursivas sin tener que devolverlo por todos lados.

Conteo de hojas. Recorre todo y suma 1 cada vez que encuentra un nodo sin hijos. No hay mejor caso posible: siempre son n visitas.

Cálculo de la altura. Este en realidad es un postorden disfrazado, porque para decidir la altura de un nodo primero necesito las alturas de todos sus hijos. Una hoja devuelve 0 y un nodo interno devuelve 1 más el máximo de lo que le devolvieron sus hijos.

```
def dfs_altura(nodo):
    if nodo is None:
        return -1
    if nodo.es_hoja():
        return 0
    return 1 + max(dfs_altura(h) for h in nodo.hijos)
```

3.3. Evidencia de ejecución

Esta es la salida real del programa sobre el árbol del punto 1, incluyendo las tres pruebas de búsqueda que pide el enunciado (un valor cerca de la raíz, uno del último nivel y uno que no existe):

```
=====
PUNTO 3 - RECORRIDOS DFS SOBRE EL ARBOL DEL PUNTO 1
=====
DFS preorden recursivo : A B E F J C G D H K L I
DFS preorden iterativo : A B E F J C G D H K L I
Coinciden los dos      : True

Total de nodos          : 12
Cantidad de hojas       : 6
Altura del arbol        : 3 (medida en aristas)

Pruebas de busqueda con DFS:
  Buscar 'B' (valor cercano a la raiz): encontrado | nodos visitados: 2
  Buscar 'L' (valor del ultimo nivel): encontrado | nodos visitados: 11
  Buscar 'Z' (valor que no existe): no encontrado | nodos visitados: 12
```

Los números confirman lo que se espera de un DFS. Buscar B cuesta 2 nodos porque es el primer hijo de la raíz. Buscar L cuesta 11 de 12 porque L está en el último nivel de la rama de más a la derecha, así que el recorrido tiene que agotar casi todo el árbol antes de llegar. Y buscar Z cuesta los 12, porque solo después de mirar el último nodo se puede afirmar que el valor no está.

También se nota que el DFS en preorden no recorre el árbol por niveles: después de A baja hasta J (que está en el nivel 3) antes de siquiera mirar a C, que está en el nivel 1. Esa es justamente la diferencia con el BFS del punto siguiente.

3.4. Análisis de complejidad

En la tabla, n es el número de nodos, h la altura del árbol y g el grado máximo:

Operación con DFS	Mejor caso	Peor caso	Espacio
Recorrer todo el árbol	$\Theta(n)$	$\Theta(n)$	$O(h)$
Buscar un valor	$O(1)$	$O(n)$	$O(h)$
Contar hojas	$\Theta(n)$	$\Theta(n)$	$O(h)$
Calcular la altura	$\Theta(n)$	$\Theta(n)$	$O(h)$

Solo la búsqueda tiene mejor y peor caso distintos, y es porque es la única que puede detenerse antes de tiempo. Si el valor está en la raíz se responde en $O(1)$; si está en la última hoja o no está, toca el árbol completo y queda en $O(n)$. Las otras tres siempre necesitan la información de todos los nodos, así que su mejor y su peor caso coinciden y por eso las escribo con Θ .

En DFS ambas versiones recursiva e iterativa tienen complejidad $O(h)$ en el caso típico, pero usan la memoria diferente y eso importa en la práctica. La recursiva se apoya en la pila del sistema, que es limitada (Python falla alrededor de mil niveles) y cada marco es pesado porque guarda parámetros y dirección de retorno. La iterativa usa una pila propia en el heap, solo almacena referencias a nodos, así que escala mejor.

Aunque la pila iterativa no solo guarda el camino, sino también los hermanos pendientes en cada nivel. Eso puede llegar a $O(h \times g)$ en lo peor, aunque siempre está limitado por $O(n)$. Cuando el árbol tiene grado bajo la diferencia es insignificante; solo importa si el grado es grande.

Ahora, lo que sí marca diferencia enorme es el balance del árbol. En uno balanceado la altura crece logarítmicamente — $h \approx \log n$ —, así que un millón de nodos son apenas veinte niveles y el DFS casi no gasta memoria. Pero si el árbol está completamente desbalanceado, donde cada nodo tiene un hijo, entonces es en realidad una lista: $h = n - 1$. El recursivo se cae por stack overflow mucho antes de terminar; el iterativo aguanta porque usa el heap.

Punto 4. Recorrido en anchura: BFS

4.1. Implementación

El BFS es el recorrido por niveles y se apoya en una cola. La idea es meter la raíz en la cola y repetir siempre lo mismo, sacar el primero que llegó, visitarlo y meter sus hijos al final. Como la cola es FIFO, los hijos quedan detrás de todos los nodos del nivel actual y por eso no se visitan hasta que ese nivel se acabe.

```
def bfs_orden_visita(raiz):
    if raiz is None:
        return []
    visitados = []
    cola = deque([raiz])
    while cola:
        actual = cola.popleft()          # sale el que lleva mas tiempo esperando
        visitados.append(actual.valor)
        for hijo in actual.hijos:
            cola.append(hijo)
    return visitados
```

Usé deque en vez de una lista normal por una razón de eficiencia que importa, en una lista de Python, pop(0) tiene que correr todos los elementos una posición y cuesta $O(n)$, así que el recorrido completo se volvería $O(n^2)$. Con deque, popleft() es $O(1)$ y el BFS se mantiene en $O(n)$.

Para agrupar por niveles, al empezar cada vuelta del ciclo, todo lo que está en la cola pertenece al mismo nivel. Entonces medimos el tamaño de la cola y saco exactamente esa cantidad de nodos; lo que entre durante esa tanda ya es del nivel siguiente.

```
def bfs_por_niveles(raiz):
    niveles = []
    cola = deque([raiz])
    while cola:
        tam = len(cola)                  # cuantos nodos hay en este nivel
        nivel_actual = []
        for _ in range(tam):
            actual = cola.popleft()
            nivel_actual.append(actual.valor)
            for hijo in actual.hijos:
                cola.append(hijo)        # estos ya son del nivel siguiente
        niveles.append(nivel_actual)
    return niveles
```

Para la búsqueda, en vez de guardar solo el nodo en la cola guardo la pareja (nodo, nivel). Así, cuando encuentro el valor, ya sé en qué nivel estaba sin tener que calcularlo aparte.

4.2. Evidencia de ejecución

```
=====
PUNTO 4 - RECORRIDO BFS SOBRE EL ARBOL DEL PUNTO 1
=====
Orden de visita BFS      : A B C D E F G H I J K L

Nivel 0: A
Nivel 1: B, C, D
Nivel 2: E, F, G, H, I
Nivel 3: J, K, L
```

Pruebas de búsqueda con BFS (los mismos valores del punto 3):

Valor buscado: B
Resultado: encontrado
Nivel del valor: 1
Nodos visitados: 2

Valor buscado: L
Resultado: encontrado
Nivel del valor: 3
Nodos visitados: 12

Valor buscado: Z
Resultado: no encontrado
Nivel del valor: no aplica
Nodos visitados: 12

Comparando con el punto anterior vemos que entre los dos recorridos. Para buscar B los dos gastan 2 nodos, porque B está cerca de la raíz por cualquier camino. Pero para buscar L el DFS gastó 11 y el BFS gastó 12, como L está en el último nivel, el BFS tiene que terminar de revisar todos los niveles anteriores antes de llegar, mientras que el DFS al menos se ahorró el nodo I. Para valores profundos el BFS es el peor de los dos, y para valores superficiales es el mejor.

El orden de visita también deja ver la diferencia: el BFS da A B C D E F G H I J K L, que es justo el orden de la tabla por niveles, mientras que el DFS daba A B E F J C G D H K L I.

4.3. Preguntas de análisis

1. ¿Por qué BFS requiere una cola? Porque una cola es FIFO y eso hace que los nodos se expandan en el mismo orden en que fueron descubiertos. Cuando visito un nodo del nivel k , sus hijos (que son del nivel $k+1$) entran al final de la cola, detrás de todos los nodos del nivel k que aún faltan. Esa es la propiedad que garantiza que el recorrido avance nivel por nivel y nunca se adelante a uno más profundo. Con cualquier otra estructura se pierde esa garantía.

2. ¿Qué sucedería si se utilizara una pila? Dejaría de ser BFS y se convertiría en un DFS. Con una pila, el último hijo que entró es el primero en salir, así que el algoritmo se hunde por una rama hasta el fondo antes de volver. De hecho ese es exactamente el DFS iterativo del punto 3: el código es casi idéntico y lo único que cambia es `pop()` por `popleft()`. Eso me pareció lo más interesante de todo el taller: es la estructura auxiliar, y no el código, la que decide el tipo de recorrido.

3. ¿Cuál es la complejidad temporal de BFS? $O(n)$. Cada nodo entra a la cola exactamente una vez y sale exactamente una vez, y esas dos operaciones son $O(1)$ con deque. Es el mismo $O(n)$ del DFS: en tiempo los dos son iguales, porque los dos tienen que tocar todos los nodos.

4. ¿Cuál es su complejidad espacial? $O(w)$, donde w es la anchura máxima del árbol, es decir el nivel que más nodos tiene. La cola nunca guarda el árbol entero, solo la frontera entre el nivel que se está procesando y el siguiente. En el árbol del punto 1 el nivel más ancho es el 2, con cinco nodos, así que la cola no pasó de tener unos pocos elementos. En el peor caso w puede ser del orden de n : en un árbol donde la raíz tiene $n - 1$ hijos, la cola termina conteniendo casi todo el árbol.

5. ¿Cuál recorrido consume más memoria en un árbol ancho: DFS o BFS? El BFS. En un árbol ancho y poco profundo, w es grande y h es pequeño: el DFS gasta $O(h)$, que es casi nada, mientras que el BFS gasta $O(w)$, que puede ser casi todo el árbol. Con un árbol de un millón de hojas colgando directamente de la raíz, el DFS usa dos niveles de pila y el BFS mete un millón de nodos en la cola. En árboles profundos y angostos pasa lo contrario, y ahí el que sufre es el DFS.

6. Si se busca el nodo menos profundo que cumpla una condición, ¿cuál conviene? El BFS. Como explora los niveles en orden creciente, el primer nodo que cumple la condición es necesariamente el de menor profundidad, y en ese momento se puede cortar la búsqueda con la certeza de que no hay uno mejor más arriba. El DFS no da ninguna garantía de ese tipo: se mete por la primera rama y puede encontrar un nodo válido en el nivel 40 cuando había uno en el nivel 2 por otra rama que todavía no ha tocado. Para estar seguro con DFS tocaría recorrer el árbol completo y quedarse con el mínimo, que es más trabajo. Esta es la misma razón por la que BFS se usa para encontrar el camino más corto en grafos sin pesos.

Punto 5. Aplicación al análisis sintáctico

La gramática del punto es la clásica de expresiones aritméticas, ya factorizada y sin recursión por la izquierda, que es lo que la hace apta para un analizador descendente:

E	->	T E'	
E'	->	+ T E'	eps
T	->	F T'	
T'	->	* F T'	eps
F	->	(E)	id

La cadena de entrada es `id + id * id`, que como lista de tokens queda `[id, +, id, *, id]`.

5.1. Construcción paso a paso del árbol

Implementé un analizador descendente recursivo, que es una función por cada no terminal. Cada función crea su propio nodo, mira el token actual para decidir qué producción aplicar y le cuelga como hijos lo que devuelvan las funciones de los símbolos del lado derecho. Los terminales se consumen de la entrada y se convierten en hojas.

El siguiente es el rastro real del analizador, con los nodos numerados en el orden exacto en que los fue creando (eso responde también el punto 5.2):

Paso	Se expande	Producción aplicada	Por qué esa	Nodos creados
1	E	$E \rightarrow T E'$	es el símbolo inicial	E(1)
2	T	$T \rightarrow F T'$	T solo tiene una producción	T(2)
3	F	$F \rightarrow id$	el token actual es id, no (	F(3), id(4)
4	T'	$T' \rightarrow \epsilon$	el siguiente token es +, no *	T'(5), ϵ (6)
5	E'	$E' \rightarrow + T E'$	el token actual sí es +	E'(7), +(8)
6	T	$T \rightarrow F T'$	única producción de T	T(9)
7	F	$F \rightarrow id$	el token actual es id	F(10), id(11)
8	T'	$T' \rightarrow * F T'$	ahora sí viene un *	T'(12), *(13)
9	F	$F \rightarrow id$	el token actual es id	F(14), id(15)
10	T'	$T' \rightarrow \epsilon$	ya no quedan tokens	T'(16), ϵ (17)
11	E'	$E' \rightarrow \epsilon$	ya no quedan tokens	E'(18), ϵ (19)

El árbol queda con 19 nodos. Lo que decide cada paso es el token que el analizador tiene al frente: en el paso 4 escoge la producción vacía para T' porque lo que sigue es un +, que no puede formar parte de una multiplicación, mientras que en el paso 8 escoge $T' \rightarrow * F T'$ porque sí aparece un *. Esa decisión con un solo token de anticipación es lo que hace que la gramática sea LL(1).

5.2. Árbol sintáctico con los nodos numerados

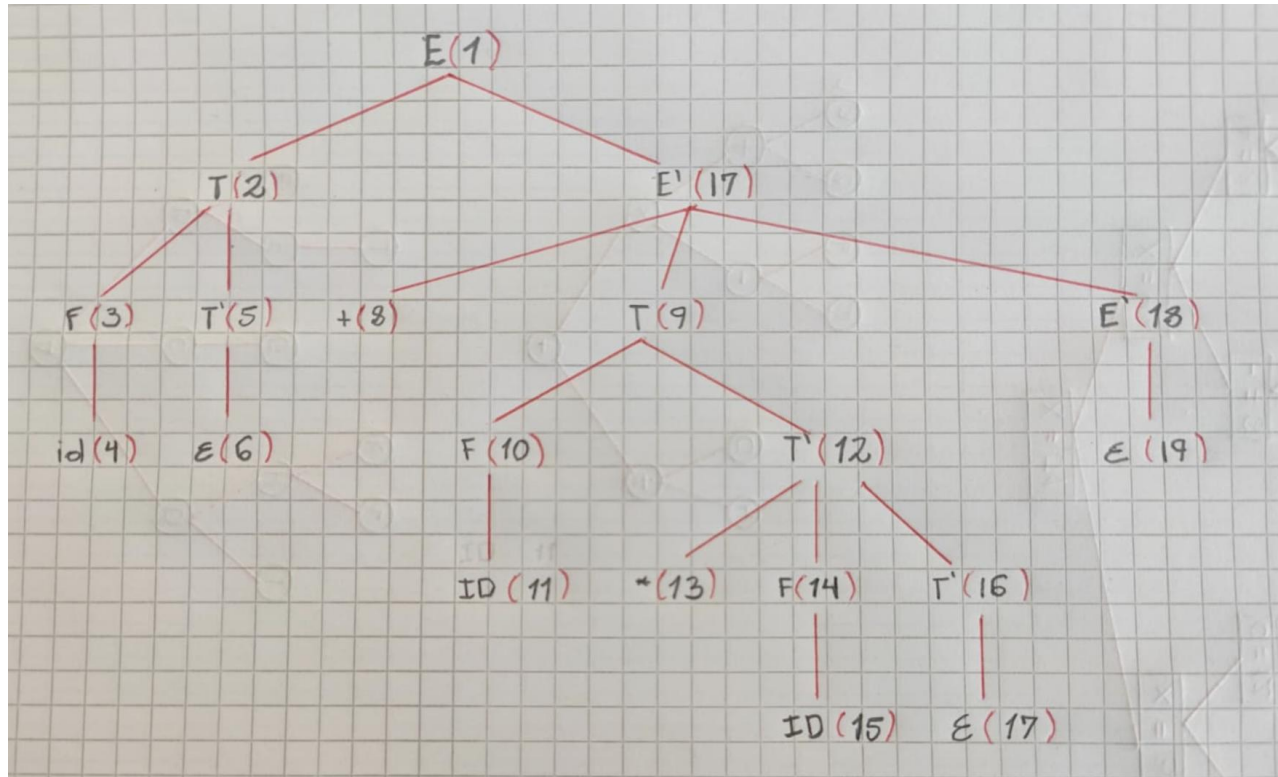


Figura 4. Árbol sintáctico de $id + id * id$. El número entre paréntesis es el orden en que el analizador creó cada nodo.

Las hojas verdes leídas de izquierda a derecha son $id, \epsilon, +, id, *, id, \epsilon, \epsilon$. Si se quitan las épsilon queda $id + id * id$, que es exactamente la cadena de entrada. Esa es la comprobación de que el árbol sí deriva la cadena.

5.3. Los tres recorridos sobre el árbol sintáctico

Esta es la salida del programa para los tres recorridos:

DFS preorden:

E(1) T(2) F(3) id(4) T'(5) ϵ (6) E'(7) +(8) T(9) F(10) id(11)
T'(12) *(13) F(14) id(15) T'(16) ϵ (17) E'(18) ϵ (19)

DFS postorden:

id(4) F(3) ϵ (6) T'(5) T(2) +(8) id(11) F(10) *(13) id(15) F(14)
 ϵ (17) T'(16) T'(12) T(9) ϵ (19) E'(18) E'(7) E(1)

BFS por niveles:

Nivel 0: E(1)
Nivel 1: T(2), E'(7)
Nivel 2: F(3), T'(5), +(8), T(9), E'(18)
Nivel 3: id(4), ϵ (6), F(10), T'(12), ϵ (19)
Nivel 4: id(11), *(13), F(14), T'(16)
Nivel 5: id(15), ϵ (17)

Los tres recorridos visitan los mismos 19 nodos, pero en órdenes completamente distintos, y cada orden sirve para algo diferente.

5.4. Qué información da cada recorrido

El preorden reconstruye la derivación. Da los nodos en el mismo orden en que el analizador tomó sus decisiones, así que leerlo es como volver a ver el análisis en cámara lenta: E se expande en T y E', T se expande en F y T', y así. Es la derivación por la izquierda de la gramática. Por eso es el recorrido que se usa para imprimir el árbol indentado o para explicar cómo se reconoció una entrada.

El postorden sirve para evaluar y generar código. Entrega cada nodo solo después de todos sus hijos, que es el orden en que se pueden calcular los atributos sintetizados: cuando llego a un no terminal, todos sus subárboles ya me dieron su valor o su fragmento de código. Es el mismo principio del punto 2, solo que aquí lo que se sintetiza no es un número sino, por ejemplo, instrucciones de código intermedio. La fase que va después del análisis sintáctico trabaja con este orden.

El BFS muestra la forma del árbol. No sirve para evaluar, porque un padre sale antes que sus hijos, pero deja ver de una sola mirada cuántos niveles tiene el árbol (aquí seis, o sea altura 5) y qué tan ancho es cada uno. Es útil para medir la profundidad del anidamiento de una expresión o para detectar entradas que van a generar árboles demasiado profundos.

5.5. Orden de creación frente al preorden

Coinciden exactamente. Si se lee el preorden de arriba, los números salen en orden perfecto: 1, 2, 3, 4, 5, ... hasta 19. Lo verifiqué en el programa comparando la lista del preorden contra esa misma lista ordenada por número de creación, y da True.

La razón no es una casualidad. Cuando una función del analizador expande un no terminal hace siempre lo mismo: primero crea el nodo de ese no terminal y después procesa los símbolos del lado derecho de izquierda a derecha, llamando a la función de cada uno. Eso es palabra por palabra la definición del preorden: visitar el padre y luego los hijos en orden. Dicho de otra manera, el analizador descendente no construye el árbol y después lo recorre: lo construye recorriéndolo en preorden. El árbol es el registro de ese recorrido.

Esto también explica por qué se llama análisis descendente: se arranca en la raíz (el símbolo inicial) y se va bajando hasta las hojas (los tokens). Un analizador ascendente, como los LR, haría lo contrario, arrancando en los tokens y construyendo los nodos internos al reducir, y ahí el orden de creación se parecería más a un postorden.

5.6. Cómo el árbol representa la precedencia de * sobre +

La precedencia no está escrita en ninguna parte del analizador, ni hay tablas de prioridades, está metida en la forma del árbol, y la forma la impone la gramática por cómo reparte el trabajo en tres niveles. E se encarga de las sumas, T de las multiplicaciones y F de los átomos, y como E está por encima de T, y T por encima de F, las sumas quedan siempre más arriba que las multiplicaciones.

En este árbol concreto se ve así: el * de la posición 13 aparece dentro de T'(12), que cuelga de T(9). Es decir, la segunda y la tercera id quedaron encerradas dentro de un mismo T, formando una sola unidad. El + de la posición 8, en cambio, está arriba en E'(7). El * quedó dos niveles más abajo que el +.

La consecuencia es directa: al evaluar en postorden, los nodos más profundos se resuelven primero, así que $id * id$ se calcula antes de que el + tenga sus dos operandos. El árbol solo puede leerse como $id + (id * id)$; no existe forma de interpretarlo como $(id + id) * id$, porque para eso el + tendría que estar debajo del

*, y la gramática nunca produce esa forma. Si se quisiera esa otra agrupación habría que escribir los paréntesis, y ahí entraría la producción $F \rightarrow (E)$, que es la que permite volver a bajar una expresión completa al nivel de los átomos y romper la precedencia a propósito.

5.7. Algoritmo para contar nodos y calcular la altura

Propongo un solo recorrido en profundidad que haga las cuatro cosas a la vez, en vez de cuatro recorridos separados. Lo hago en postorden porque la altura de un nodo depende de la de sus hijos, y de paso aprovecho la visita para ir clasificando el nodo. El pseudocódigo es:

```
ALGORITMO estadísticas(nodo)
  si nodo es terminal:
    si nodo.símbolo == epsilon:  epsilon <- epsilon + 1
    si no:                       terminales <- terminales + 1
  si no:
    no_terminales <- no_terminales + 1

  si nodo no tiene hijos:
    devolver 0 // es una hoja, altura 0

  maxima <- 0
  para cada hijo de nodo:
    maxima <- max(maxima, estadísticas(hijo))
  devolver 1 + maxima // la altura del padre
```

La implementación real en Python es esta, usando una función interna para no tener que ir pasando los acumuladores por todos lados:

```
def estadísticas_arbol_sintactico(nodo):
    resultado = {"terminales": 0, "no_terminales": 0, "epsilon": 0, "altura": 0}

    def recorrer(n):
        if n.terminal:
            if n.valor == "eps":
                resultado["epsilon"] += 1 # las eps las cuento aparte
            else:
                resultado["terminales"] += 1
        else:
            resultado["no_terminales"] += 1

        if not n.hijos:
            return 0 # hoja -> altura 0
        return 1 + max(recorrer(h) for h in n.hijos)

    resultado["altura"] = recorrer(nodo)
    return resultado
```

Cuesta $O(n)$ en tiempo, porque visita cada nodo una vez y hace trabajo constante en cada uno, y $O(h)$ en espacio por la recursión. Hacerlo en un solo recorrido en vez de cuatro no cambia el orden de complejidad, pero sí divide entre cuatro el número real de visitas. La salida sobre el árbol de $id + id * id$ es:

```
Estadísticas del árbol sintactico (punto 5.7):
Nodos no terminales      : 11
Nodos terminales        : 5
Producciones vacías (eps): 3
Altura del árbol         : 5
Total de nodos           : 19
```

Los números cuadran con el dibujo. Los 11 no terminales son E, T, F, T', E', T, F, T', F, T' y E'. Los 5 terminales son las tres id, el + y el *, que es exactamente la cantidad de tokens de la entrada, como tenía que ser. Las 3 épsilon vienen de las tres veces que el analizador escogió una producción vacía (dos veces para T' y una para E'). Y $11 + 5 + 3 = 19$, que es el total de nodos.

Comparación final entre DFS y BFS

Criterio	DFS	BFS
Estructura auxiliar	Pila (LIFO). Puede ser explícita o la pila de llamadas de la recursión.	Cola (FIFO), normalmente un deque para que sacar el primero sea $O(1)$.
Orden de exploración	En profundidad: se hunde por una rama hasta el fondo y luego retrocede.	Por niveles: termina un nivel completo antes de pasar al siguiente.
Complejidad temporal	$O(n)$	$O(n)$
Complejidad espacial	$O(h)$, la altura del árbol.	$O(w)$, la anchura máxima del árbol.
Conveniente para evaluar expresiones	Sí. El postorden entrega los operandos antes que el operador, que es justo lo que se necesita.	No. Un padre sale antes que sus hijos, así que al llegar al operador sus operandos aún no existen.
Conveniente para recorrer por niveles	No. Mezcla nodos de distintos niveles y para agruparlos habría que guardar la profundidad aparte.	Sí. Es su comportamiento natural y agruparlos solo cuesta medir el tamaño de la cola.
Comportamiento en árboles profundos	Malo. h crece y la pila también; con recursión hay riesgo de stack overflow.	Bueno. Los niveles son angostos, así que la cola se mantiene pequeña.
Comportamiento en árboles anchos	Bueno. h es pequeña y la pila casi no crece.	Malo. Un nivel muy ancho obliga a tener casi todo el árbol en la cola a la vez.

Resumiendo lo que más me quedó claro de la tabla: en tiempo los dos son iguales, $O(n)$, porque los dos tienen que mirar todos los nodos. La diferencia real está en la memoria y en el orden. DFS gasta según la altura y BFS según la anchura, así que son complementarios: el árbol que hace sufrir a uno es cómodo para el otro. Y el orden decide para qué sirve cada uno: si lo que se quiere es calcular algo de abajo hacia arriba, DFS en postorden; si lo que se quiere es lo más cercano a la raíz, BFS.